

PRÁCTICA 5-B

Ingeniería del Software II
(Pedro Cuesta y Hugo Silva)

Análisis y Diseño de Casos de Prueba

Para esta parte de la práctica, debemos volver a diseñar casos de prueba utilizando caja negra y caja blanca, e implementarlos utilizando el framework JUnit. Esta vez se diseñarán casos sobre una estructura de datos. Para ello, partimos de una interfaz `IConjuntoOrdenado` implementada en una clase `ConjuntoOrdenado`, sobre la que tendremos que diseñar los casos, probarlos, y verificar el correcto funcionamiento de la clase en función de la documentación dada.

1. DISEÑO DE CASOS DE PRUEBA. CAJA NEGRA

Empezaremos diseñando los casos de prueba de caja negra, apoyándonos en la documentación de la interfaz.

Caja Negra:

Función `get`:

Parámetro	Clases válidas	Clases no válidas
<code>index</code>	-[0, <code>lista.size</code>)	- <code>index < 0</code> - <code>index >= lista.size</code>
<code>lista</code>	-lista con elementos	-lista vacía

Función `add`:

Parámetro	Clases válidas	Clases no válidas
<code>elemento</code>	-elemento que no está en la lista -elemento que está en la lista	null
<code>lista</code>	-lista vacía -lista con elementos	

Función remove:

Parámetro	Clases válidas	Clases no válidas
index	-[0, lista.size)	-index < 0 -index >= lista.size
lista	-lista con elementos	-lista vacía

Función size:

Parámetro	Clases válidas	Clases no válidas
lista	-lista vacía -lista con elementos	

Función clear:

Parámetro	Clases válidas	Clases no válidas
lista	-lista vacía -lista con elementos	

La lista de valores junto con el análisis de valores límite no se han escrito, debido a la dependencia entre el estado de la estructura de datos y los valores a utilizar para los métodos. Por este motivo, a continuación se muestran directamente los casos de prueba válidos y no válidos. Aun así, se ha tenido en cuenta el AVL para realizar los casos de prueba. Además de analizar los valores en los parámetros de entrada de los métodos, es necesario tener en cuenta el estado de la lista y cómo es modificada tras ejecutar algún método. Hemos de tener en cuenta la lista en tres estados diferentes: Lista vacía, lista con 1 elemento y lista con varios elementos. A la hora de diseñar los casos de prueba, deberemos asegurarnos de que el funcionamiento de los métodos es el esperado para todos los estados de la lista. Así, teniendo de clase válida "lista con elementos", deberemos probar el método asociado en los estados "Lista con 1 elemento" y "Lista con varios elementos".

CASOS DE PRUEBA VÁLIDOS	
get	1. [10], 0: 10. 2. [10, 20, 30], 1: 20. 3. [10, 20, 30], 2: 30.
add	1. [], 30: true, ([30]) 2. [30], 10: true ([10, 30]) 3. [10, 30], 20: true ([10, 20, 30]) 4. [10, 20, 30], 10: false, ([10, 20, 30])
remove	1. [20, 40, 50], 1: 40, ([20, 50]) 2. [20, 50], 1: 50, ([20]) 3. [20], 0: 20, ([])
size	1. []: 0 2. [10]: 1 3. [10, 20, 30]: 3
clear	1. []: ([]) 2. [10]: ([]) 3. [10, 20, 30]: ([])

Get considera que una lista vacía no es válida, por lo que solo debemos comprobar el funcionamiento del método para una lista con 1 elemento y una con varios elementos. Es necesario tener en cuenta el valor del índice: el rango válido es $0 \leq \text{índice} < \text{lista.size()}$, por lo que, aplicando AVL, deberemos considerar los índices 0 y $\text{lista.size()} - 1$, además de un índice intermedio. Los valores de los índices están “hardcodeados” en los tests por simplicidad (2 en vez de $\text{lista.size()} - 1$).

Para el método add jugamos con los estados de la lista y las posiciones sobre las que añadir para hacer menos casos de prueba. Comenzamos con una lista vacía, añadimos un elemento al principio de la lista y cambia su estado a “Lista con 1 elemento”. Después añadimos otro elemento al final de la lista, y de nuevo, el estado cambia a “Lista con varios elementos”. Por último, añadimos un elemento en medio, e intentamos añadir un elemento repetido para verificar que no se introduce.

Con el método remove pasa algo similar: partimos de una lista con 3 elementos, sobre la que iremos eliminándolos hasta quedarnos con una lista vacía. De nuevo, la lista pasa por todos los estados.

Para size es necesario, de nuevo, comprobar todos los estados, por lo que se desarrollan 3 casos de prueba distintos.

Clear es un método de tipo void, por lo que la única forma que tenemos de verificar el funcionamiento es a través del estado de la lista. Sea cual sea el estado inicial de la lista antes de llamar al método, tras invocarlo, el estado será “Lista vacía” siempre. En los tests se comprobará el tamaño de la lista.

CASOS DE PRUEBA NO VÁLIDOS	
get	1. [], 0: IndexOutOfBoundsException. 2. [10], -1: IndexOutOfBoundsException 3. [10, 20], -4: IndexOutOfBoundsException 4. [10, 20], 2: IndexOutOfBoundsException 5. [10, 20], 7: IndexOutOfBoundsException
add	1. [], null: NullPointerException
remove	1. [], 0: IndexOutOfBoundsException. 2. [10], -1: IndexOutOfBoundsException 3. [10, 20], -4: IndexOutOfBoundsException 4. [10, 20], 2: IndexOutOfBoundsException 5. [10, 20], 7: IndexOutOfBoundsException
size	-
clear	-

Al igual que antes, aplicando AVL en los índices, debemos considerar los límites en los rangos no válidos. Tanto en get como en remove tenemos los mismos tests, ya que los rangos son iguales. Para el rango $\text{index} < 0$, comprobamos en -1 y en otro índice inferior, y para el otro rango $\text{index} \geq \text{lista.size()}$, comprobamos en lista.size() y en algo superior. De nuevo, los índices están “hardcodeados”.

Size y clear no tienen clases no válidas, por lo que es razonable que no hayan casos de prueba no válidos.

En add tan solo debemos considerar que el elemento a añadir sea igual a null, en cuyo caso se debería lanzar un NullPointerException.

2. IMPLEMENTACIÓN Y EJECUCIÓN DE LOS TESTS CON JUNIT. DETECCIÓN Y CORRECCIÓN DE ERRORES

Con esto tenemos cubiertos los casos de prueba de caja negra. Lo siguiente será implementar los casos en un proyecto Maven usando JUnit. Habrá un método de test por método original en la clase, es decir, 5 métodos, sin contar el constructor.

Al terminar la implementación, ejecutamos los tests en nuestra máquina para verificar el comportamiento de los métodos. Es aquí donde observamos un error claro: Según la implementación, add debería retornar false en caso de que se estuviera intentando añadir un elemento ya presente en la lista. Sin embargo, el test falla. Si observamos la Imagen 1, el valor esperado es FALSE debido al AssertFalse, y sin embargo, el valor devuelto es TRUE. Si además vemos la Imagen 2, se nos indica otro fallo en los tests: Al verificar el estado de la lista después de añadir un elemento en el método add (que consiste en comprobar si el orden de los elementos es el esperado, ordenados de manera natural, según la documentación), todo parece indicar que no se ha ordenado la lista como debería. Es necesario revisar el código del método.

La Imagen 3 muestra cómo está implementado el método add de la clase ConjuntoOrdenado, y no se observa en ningún punto la posibilidad de que el método devuelva FALSE. Si analizamos con un poco más de detalle el código, llegamos a la conclusión de que add no trata los repetidos, los añade sin comprobar si ya están en la lista. Debemos corregir este fallo. Para ello, simplemente añadimos una comprobación que verifique si la lista contiene al elemento.

En cuanto al segundo error detectado, el fallo está en el while, donde se comprueba el valor del elemento con los elementos de la lista para ver en qué posición se añade. La implementación es tal que ordena los elementos de mayor a menor, es decir, al contrario del orden esperado. La solución propuesta consiste simplemente en cambiar la condición a `elemento.compareTo(lista.get(indice)) > 0`, asegurando el comportamiento esperado a la hora de reorganizar la lista al añadir un elemento en ella. La Imagen 4 muestra el método actualizado al completo.

Tras esto, volvemos a ejecutar los tests, y ahora sí, todos pasan correctamente. No se observan más fallos de funcionalidad en los métodos, como se ilustra en la Imagen 5.

3. COBERTURA DE LOS TESTS

En Visual Studio, hemos hecho un “Run Test with Coverage” para comprobar la cobertura de los tests. En la imagen 5 el Test Coverage es del 100%, es decir, hemos cubierto todas las casuísticas de todos los métodos, la cobertura es total. No será necesario diseñar nuevos casos de prueba.

Con esto hemos completado la tarea: Hemos diseñado casos de prueba de caja negra, los hemos ejecutado, hemos detectado fallos en la implementación de la clase, los hemos corregido, y hemos conseguido un Coverage en los tests del 100%.

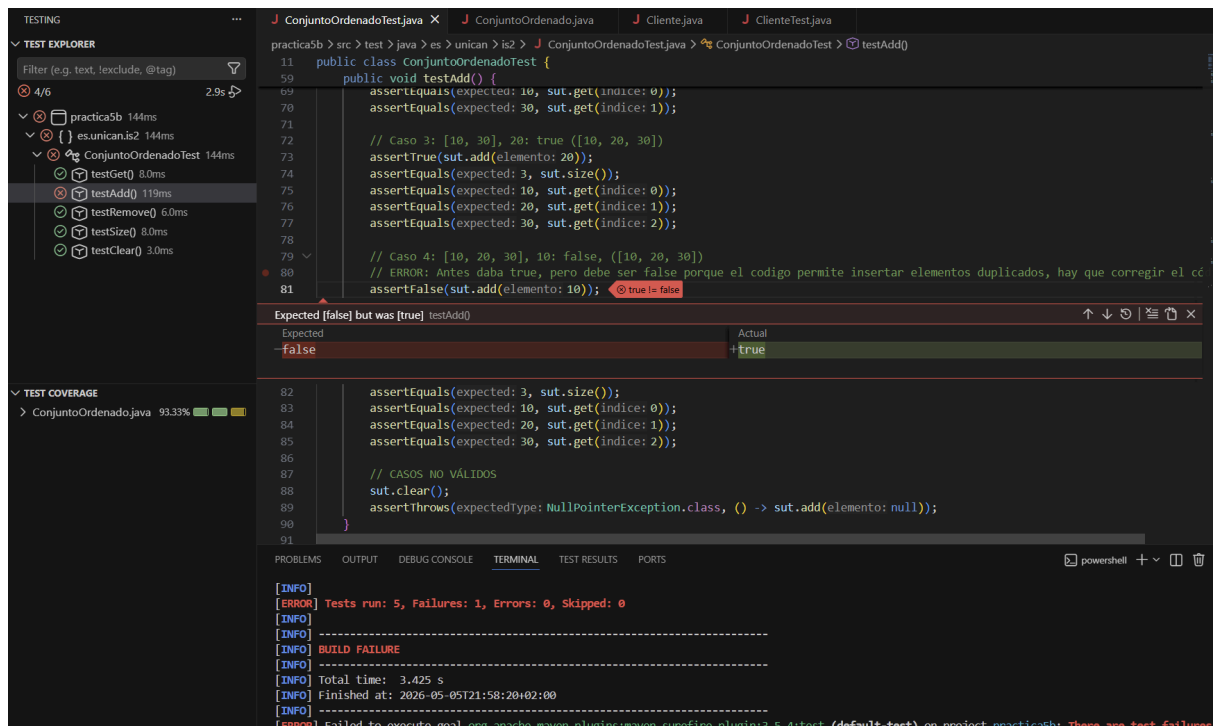


Imagen 1: Error en test por inserción de duplicados

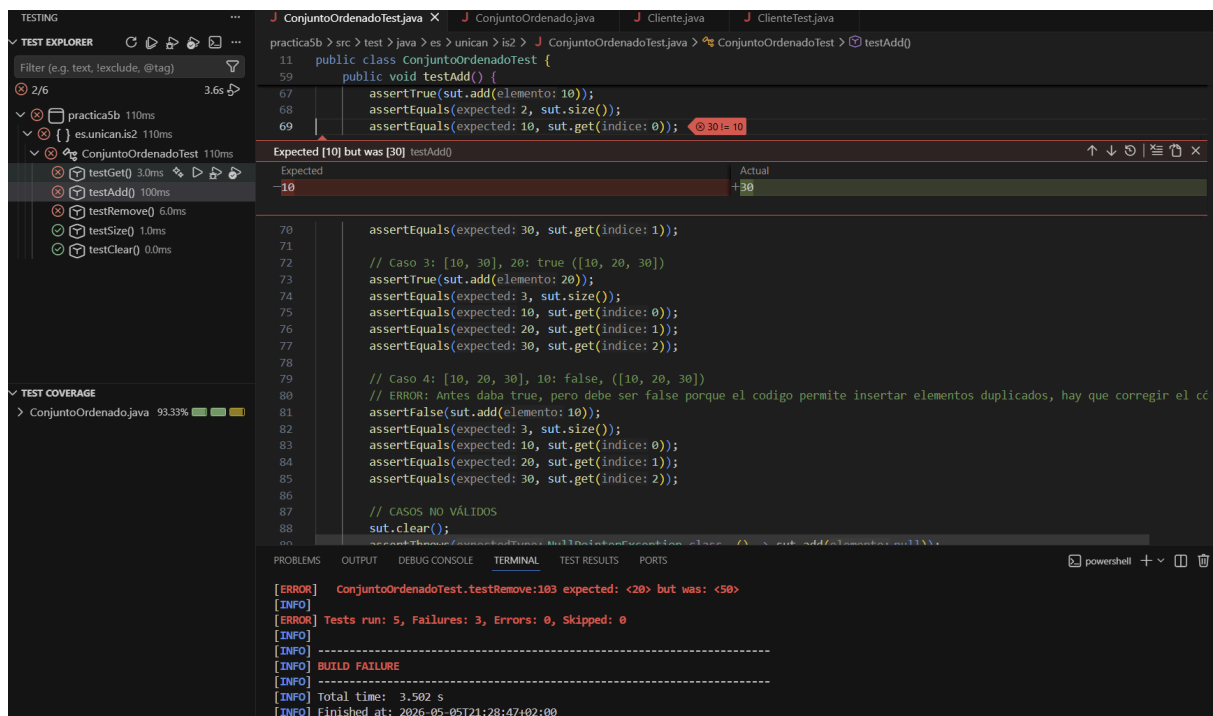


Imagen 2: Error en test por ordenación no natural de la lista

```

18     public boolean add(E elemento) {
19         int indice = 0;
20         if (elemento==null)
21             throw new NullPointerException();
22
23         if (lista.size() != 0) {
24             while (indice < lista.size() && elemento.compareTo(lista.get(indice)) < 0) {
25                 indice++;
26             }
27         }
28         lista.add(indice, elemento);
29         return true;
30     }
31

```

Imagen 3: Método add sin corregir

```

17
18     public boolean add(E elemento) {
19         int indice = 0;
20         if (elemento==null)
21             throw new NullPointerException();
22
23         // DEPURACIÓN DE CÓDIGO, tras ejecutar caja negra. Si ya existe, devolvemos false y no lo añadimos
24         if (lista.contains(elemento)) {
25             return false;
26         }
27
28         // Cambiamos el menor que por mayor que para que la lista se ordene de manera natural (de menor a mayor)
29         if (lista.size() != 0) {
30             while (indice < lista.size() && elemento.compareTo(lista.get(indice)) > 0) {
31                 indice++;
32             }
33         }
34         lista.add(indice, elemento);
35         return true;
36     }

```

Imagen 4: Método add corregido

The screenshot displays an IDE interface with the following components:

- TESTING** tab selected.
- TEST EXPLORER** on the left, showing a filter and a list of tests:
 - practica5b 126ms
 - es.unican.is2 126ms
 - ConjuntoOrdenadoTest 126ms
 - testGet() 4.0ms
 - testAdd() 107ms
 - testRemove() 8.0ms
 - testSize() 5.0ms
 - testClear() 2.0ms
- TEST COVERAGE** section below, showing 100.00% coverage for ConjuntoOrdenado.java.
- Code Editor** showing `ConjuntoOrdenadoTest.java` with the following content:

```

11 public class ConjuntoOrdenadoTest {
59     public void testAdd() {
60
61         // Caso 2: [30], 10: true ([10, 30])
62         assertTrue(sut.add(elemento: 10));
63         assertEquals(expected: 2, sut.size());
64         assertEquals(expected: 10, sut.get(indice: 0));
65         assertEquals(expected: 30, sut.get(indice: 1));
66
67         // Caso 3: [10, 30], 20: true ([10, 20, 30])
68         assertTrue(sut.add(elemento: 20));
69         assertEquals(expected: 3, sut.size());
70         assertEquals(expected: 10, sut.get(indice: 0));
71         assertEquals(expected: 20, sut.get(indice: 1));
72         assertEquals(expected: 30, sut.get(indice: 2));
73
74         // Caso 4: [10, 20, 30], 10: false, ([10, 20, 30])
75         // ERROR: Antes daba true, pero debe ser false porque el código permite insertar elementos duplicados, hay que corregir el código
76         assertFalse(sut.add(elemento: 10));
77         assertEquals(expected: 3, sut.size());
78         assertEquals(expected: 10, sut.get(indice: 0));
79         assertEquals(expected: 20, sut.get(indice: 1));
80         assertEquals(expected: 30, sut.get(indice: 2));
81
82         // CASOS NO VÁLIDOS
83         sut.clear();
84         sut.add(elemento: null);
85         sut.add(elemento: null);
86     }
87 }
88

```
- Terminal** at the bottom showing the test results:

```

[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 3.184 s
[INFO] Finished at: 2026-05-05T21:30:53+02:00

```

Imagen 5: Resultado de los tests tras la corrección de código